1. What is Dart?

Dart is a programming language developed by Google.

Dart is commonly used with Flutter to develop mobile applications for Android and iOS, as well as web and desktop applications.

For example:

- **Dart** is the programming language.
- **Flutter** is a framework that uses Dart to build user interfaces.
- **SDK** is a software development kit that provides reusable functionality to other developers.

What can we develop using Dart?

- Mobile applications using Flutter.
- Reusable libraries and packages.
- SDKs that other applications can integrate.
- Networking and API communication components.
- Utility functions, data-processing components, and other reusable software.

2. Understanding Dart's basic program structure

Concept	Meaning
`main()`	Starting point of the program
`print()`	Displays output
`{ }`	Defines a block of code
`;`	Ends a statement
`//`	Begins a single-line comment

3. Variables in Dart

A **variable** is a container used to store information.

Data type	Used for	Example
`String`	Text	`'Hello'`
`int`	Whole numbers	`10`
`double`	Decimal numbers	`4.5`
`bool`	True or false	`true`
`List`	A collection of values	`['Login', 'Logout']`
`Map`	Key-value pairs	`{'name': 'Testing'}`

What is `var`?

Dart can automatically determine a variable's type from its initial value.

```
void main() {
  var name = 'Testing';
  var age = 25;

  print(name);
  print(age);
}
```

Here, Dart infers that `name` is a **`String`** and **`age`** is an **`int`**.

4. Understanding `final` and `const`

These keywords are especially useful when developing SDKs because some values should not be reassigned.

A. Using `final`

A `final` variable can be assigned only once.

```
void main() {
  final appName = 'My SDK';

  print(appName);
}
```

You cannot later assign a different value to `appName`.

B. Using `const`

A `const` variable represents a compile-time constant.

```
void main() {
  const sdkVersion = '1.0.0';

  print(sdkVersion);
}
```

Feature	`final`	`const`
Can be reassigned?	No	No
Value must be known at compile time?	No	Yes
Example	A value obtained from a configuration file	A fixed constant defined in code

5. Operators in Dart

Operators perform operations on values

A. Arithmetic operators

```
void main() {
  int a = 10;
  int b = 3;

  print(a + b); //Addition
  print(a – b); //Subtraction
  print(a * b); //Multiplication
  print(a / b); //Division
  print(a % b); //Remainder
}
```

B. Comparison operators

These operators compare two values.

```
void main() {
  int version = 2;

  print(version == 2); //true
  print(version != 1); //true
  print(version > 1); //true
  print(version < 5); //true
}
```

Remember:

- = assigns a value.
- == checks whether two values are equal.
- != checks whether two values are different.

C. Logical operators

Logical operators combine conditions.

```
void main() {
  bool isInitialized = true;
  bool isEnabled = true;

  print(isInitialized && isEnabled); //true
  print(isInitialized || isEnabled); //true
  print(!isInitialized); //false
}
```

Operator	Meaning
&&	AND: both conditions must be true
!	NOT: reverses a Boolean value

6. Conditional statements: `if`, `else if`, and `else`

Conditional statements allow your program to make decisions.

```
void main() {
  int version = 2;

  if (version == 1) {
    print('Version 1');
  } else if (version == 2) {
    print('Version 2');
  } else {
    print('Unknown version');
  }
}
```

Output: `Version 2`

This concept will help us implement SDK configuration validation, error handling, and feature checks.

7. Loops in Dart

Loops allow you to execute a block of code repeatedly.

A. `for` loop

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print(i);
  }
}
```

Output:

```
1
2
3
4
5
```

B. `for-in` loop

```
void main() {
  List<String> events = [
    'app_opened',
    'login',
    'logout'
  ];

  for (String event in events) {
    print(event);
  }
}
```

Output:

```
app_opened
login
logout
```

A `for-in` loop processes each item in a collection.

SDK connection: This could be useful when processing a collection of events waiting to be sent to a server.

8. Functions in Dart

A **function** is a reusable block of code that performs a particular task.

For example, instead of writing the same code repeatedly, you can create a function and call it whenever required.

Example 1: A simple function

```
void greet() {
  print('Welcome to Dart');
}

void main() {
  greet();
  greet();
}
```

```
Welcome to Dart
Welcome to Dart
```

Example 2: A function with parameters

```
void greetUser(String name) {
  print('Hello, $name');
}

void main() {
  greetUser('Mehul');
  greetUser('Rahul');
}
```

```
Hello, Mehul
Hello, Rahul
```

The $name syntax inserts the variable's value into the string. This is called **string interpolation**.

Example 3: A function that returns a value

```
int add(int a, int b) {
  return a + b;
}

void main() {
  int result = add(10, 20);

  print(result);
}
```

```
30
```

`int`: The function returns an integer.

`add`: The function name.

`int a, int b`: The two parameters.

`return a + b`: Returns the result.

9. Collections: `List` and `Map`

A. List

A `List` stores multiple values in an ordered collection.

```
void main() {
  List<String> events = [
    'app_opened',
    'login',
    'logout'
  ];

  print(events[0]);
  print(events[1]);
  print(events.length);
}
```

```
app_opened
login
3
```

Remember:

- List indexing starts at `0`.
- `events[0]` accesses the first item.
- `events.length` returns the number of items.

You can add an item:

```
events.add('payment');
```

Now the list contains four events.

B. Map

A `Map` stores data as key-value pairs.

```
void main() {
  Map<String, dynamic> user = {
    'name': 'Rahul',
    'age': 25,
    'isActive': true
  };

  print(user['name']);
  print(user['age']);
  print(user['isActive']);
}
```

```
Rahul
25
true
```

Here:

- `String` means the keys must be strings.
- `dynamic` allows the values to have different types.
- `'name'`, `'age'`, and `'isActive'` are keys.

10. Classes and Objects

Classes and objects are fundamental to designing a reusable SDK.

A class defines the properties and behavior of something. An object is an instance of that class.

```
class User {
  String name;
  int age;

  //Create a constructor
 //A constructor initializes a new object.
  User(this.name, this.age);

  //A method is a function associated with a class or object.
  void displayInfo() {
    print('Name: $name');
    print('Age: $age');
  }
}

void main() {
  User user1 = User('Rahul', 25);

  user1.displayInfo();
}
```

```
Name: Rahul
Age: 25
```

11. Null safety in Dart

Dart has a feature called **null safety**, which helps prevent errors caused by missing values.

```
String name = 'Rahul';
```

It must contain a string and cannot be assigned `null`.

```
String? userId;
```

The `?` means that `userId` can contain either a string or `null`.

12. Asynchronous programming: `async` and `await`

`Imagine that your SDK needs to send application data to an API. The request might take some time to complete.`

You don't want your program to assume the operation has finished before it actually has.

Dart supports asynchronous programming through features such as `Future`, `async`, and `await`.

```
Future<void> fetchData() async {
  print('Fetching data...');

  await Future.delayed(
    Duration(seconds: 2),
  );
```

```
Fetching data...
Data received
Program completed
```

```
  print('Data received');
}

Future<void> main() async {
  await fetchData();

  print('Program completed');
}
```

There is a two-second delay before `Data received` appears.

Important keywords

Keyword	Meaning
`Future`	Represents a result that will become available later
`async`	Marks a function as asynchronous
`await`	Waits for an asynchronous operation to complete before continuing that function